Das (be)greifbare Smart Home

Konzeption und Konstruktion einer modularen 12V-IoT-Lernumgebung für die Digitale Grundbildung

MASTERARBEIT

zur Erlangung des Mastergrades
Master of Education MEd

an der Universität Salzburg
School of Education

Gutachterin: Mag. Corinna Hörmann, PhD
Unterrichtsfach der Masterarbeit:
Informatik und Digitale Grundbildung

eingereicht von
Johannes Baischer, BEd.

Salzburg, Oktober 2026

Inhaltsverzeichnis

1 Vorwort V

2 Zusammenfassung VI

3 Abstract VII

4 Einleitung 1
- 4.1 Ausgangslage und Motivation . . . 1
- 4.2 Problemstellung . . . 1
- 4.3 Zielsetzung und Aufbau der Arbeit . . . 1

5 Fachdidaktischer und theoretischer Rahmen 2
- 5.1 Curriculare Verortung . . . 2
- 5.2 Interdisziplinarität . . . 2
  - 5.2.1 Bezüge zur Physik . . . 2
  - 5.2.2 Bezüge zur Mathematik . . . 2
- 5.3 Physical Computing und Tangible Interfaces . . . 2
- 5.4 Prinzipien didaktischer Reduktion . . . 2

6 Didaktisches Design der Smart-Home-Lernumgebung 3
- 6.1 Hardware-Konzept und Sicherheit . . . 3
- 6.2 Autonome Infrastruktur . . . 3
- 6.3 Modularität und Haptik . . . 3

7 Das Smart-Home im Unterricht: Konzeptionelle Lernszenarien 4
- 7.1 Methodischer Zugang . . . 4
- 7.2 Modul 1: Strom und Elektronik im Smart-Home . . . 4
- 7.3 Modul 2: Smart-Home und Umwelt . . . 4
- 7.4 Modul 3: Automatisierung und Logik . . . 4
- 7.5 Modul 4: Netzwerke und IoT . . . 4
- 7.6 Modul 5: Smart-Home und Datenschutz . . . 4

8 Kritische Reflexion und Limitationen 5
- 8.1 Reflexion des Entwicklungsprozesses . . . 5
- 8.2 Skalierbarkeit im Schulalltag . . . 5
- 8.3 Didaktische Hürden . . . 5

9 Vergleich mit bestehenden IoT-Lernumgebungen 6
9.1 Vergleichskriterien 6
9.2 LEGO Mindstorms 6
9.3 Calliope mini 6
9.4 micro:bit 6
9.5 Arduino Starterkits 6
9.6 Einordnung der eigenen Lernumgebung 6

10 Fazit und Ausblick 7
10.1 Fazit 7
10.2 Ausblick 7

11 KI-Erklärung 8

12 Anhang iv
12.1 Technisches Handbuch iv

Todo list

Remove TODO List IV

Vorwort schreiben V

Keywords ergänzen V

Kapitelthema und Forschungsfrage festlegen. 1

IoT im Alltag der Schüler:innen begründen. 1

Technische und datenschutzbezogene Probleme darstellen. 1

Ziele und Kapitelaufbau knapp erläutern. 1

Theoretische Grundlagen und Fachdidaktik ausarbeiten. 2

Lehrplanbezug und Kompetenzen zuordnen. 2

Fächerübergreifende Bezüge herausarbeiten. 2

Strom, Spannung und Leistung einordnen. 2

Logik und Messdaten mathematisch verknüpfen. 2

Tangible Interfaces lerntheoretisch begründen. 2

Low Floor und High Ceiling erklären. 2

Didaktische Begründung des Systemdesigns entwickeln. 3

12V-SELV, Sicherheit und ESP32 begründen. 3

Lokale Infrastruktur und Cloudfreiheit begründen. 3

Modularität und haptisches Lernen erklären. 3

Fünf konzeptionelle Unterrichtsmodule ausarbeiten. 4

Forschendes und projektbasiertes Lernen beschreiben. 4

Stromkreise, Wasser-Modell und Relais vermitteln. 4

Energieeffizienz, Sensorik und Vampirstrom behandeln. 4

Bedingungen, Zustände und Requests erklären. 4

IP-Adressen, Subnetze und Router vermitteln. 4

Cloud, Lokalbetrieb und Datensouveränität vergleichen. 4
Entwicklung, Skalierung und Grenzen kritisch reflektieren. 5
Technische und didaktische Kompromisse dokumentieren. 5
Kosten, Zeitaufwand und Wartung einschätzen. 5
Fehlvorstellungen und Unterstützungsmaßnahmen beschreiben. 5
Bestehende IoT-Lernumgebungen systematisch vergleichen. 6
Technische und didaktische Kriterien festlegen. 6
Konstruktion, Programmierung und Kosten vergleichen. 6
Einstieg, Sensorik und Curriculumpassung vergleichen. 6
Einstieg, Sensorik und Curriculumpassung vergleichen. 6
Offenheit, Hardwarezugang und Sicherheit vergleichen. 6
Eigene Lernumgebung im Kriterienraster einordnen. 6
Ergebnisse zusammenfassen und Weiterentwicklungen skizzieren. 7
Zentrale Ergebnisse und Beitrag zusammenführen. 7
Sensoren, KI und Oberstufe als Perspektiven skizzieren. 7

move TO-
O List

Mögliche Ergänzungen und Fragen

- Aufbau der Arbeit wirklich nur grob strukturiert und absolut nicht final

1 Vorwort

...

Johannes Baischer
Salzburg, 26. Oktober 2026

(..., ..., ..., ..., ...)

Vorwort schreiben

Keywords ergänzen

2 Zusammenfassung

...

...

3 Abstract

...

...

4 Einleitung

4.1 Ausgangslage und Motivation

4.2 Problemstellung

4.3 Zielsetzung und Aufbau der Arbeit

Kapitelthema und Forschungsfrage festlegen.

IoT im Alltag der Schüler:innen begründen.

Technische und datenschutzbezogene Problem darstellen.

Ziele und Kapitelaufbau knapp erläutern.

5 Fachdidaktischer und theoretischer Rahmen

eoretische undlagen d Fachdi- ktik ausar- iten.

5.1 Curriculare Verortung

hrplan- zug und mpetenzen ordnen.

5.2 Interdisziplinarität

5.2.1 Bezüge zur Physik

cherüber- eifende züge her- sarbeiten.

5.2.2 Bezüge zur Mathematik

rom, Span- ng und istung ein- dnen.

5.3 Physical Computing und Tangible Interfaces

gik und essdaten athema- ch ver- üpfen.

5.4 Prinzipien didaktischer Reduktion

ngible In- rfaces lern- eoretisch gründen.

w Floor d High Cei- g erklären.

6 Didaktisches Design der Smart-Home-Lernumgebung

Didaktische Begründun des System designs ent wickeln.

6.1 Hardware-Konzept und Sicherheit

6.2 Autonome Infrastruktur

12V-SELV, Sicherheit und ESP32 begründen.

6.3 Modularität und Haptik

Lokale Infr struktur ur Cloudfreihe begründen.

Modularitä und hapti- sches Lern erklären.

7 Das Smart-Home im Unterricht: Konzeptionelle Lernszenarien

nf kon-
ptionelle
terrichts-
odule aus-
beiten.

7.1 Methodischer Zugang

rschendes
d projekt-
siertes
rnen be-
hreiben.

7.2 Modul 1: Strom und Elektronik im Smart-Home

romkrei-
Wasser-
odell und
lais vermit-
n.

7.3 Modul 2: Smart-Home und Umwelt

ergieeffi-
enz, Sen-
rik und
mpirstrom
handeln.

7.4 Modul 3: Automatisierung und Logik

dingungen,
stände
d Requests
klären.

7.5 Modul 4: Netzwerke und IoT

-Adressen,
bnetze
d Router
rmitteln.

7.6 Modul 5: Smart-Home und Datenschutz

oud, Lo-
lbetrieb
d Daten-
uveränität
rgleichen.

8 Kritische Reflexion und Limitationen

8.1 Reflexion des Entwicklungsprozesses

8.2 Skalierbarkeit im Schulalltag

8.3 Didaktische Hürden

Entwicklun
Skalierung
und Grenze
kritisch re-
flektieren.

Technische
und didakt
sche Kompr
misse doku
mentieren.

Kosten, Zei
aufwand ur
Wartung ei
schätzen.

Fehlvorstel
lungen und
Unterstüt-
zungsmaß-
nahmen be
schreiben.

9 Vergleich mit bestehenden IoT-Lernumgebungen

stehende T-rnumgebungen stematisch rgleichen.

chnische d didakti-he Kriteri- festlegen.

nstruktion, ogrammie-ng und sten ver-eichen.

nstieg, Sen-rik und rriculum-ssung ver-eichen.

nstieg, Sen-rik und rriculum-ssung ver-eichen.

fenheit, rdware-gang und cherheit rgleichen.

gene Lern-ngebung im iterienras-r einord-n.

9.1 Vergleichskriterien

9.2 LEGO Mindstorms

9.3 Calliope mini

9.4 micro:bit

9.5 Arduino Starterkits

9.6 Einordnung der eigenen Lernumgebung

10 Fazit und Ausblick

10.1 Fazit

10.2 Ausblick

Ergebnisse zusammenfassen und Weiterentwicklungen skizzieren.

Zentrale Ergebnisse und Beitrag zusammenführen.

Sensoren, K und Oberstufe als Perspektiven skizzieren.

11 KI-Erklärung

Während der Erstellung dieser Arbeit wurden generative Werkzeuge der Künstlichen Intelligenz (KI), darunter ChatGPT, Gemini und DeepL, genutzt, um die sprachliche Überarbeitung zu unterstützen, die Lesbarkeit zu verbessern sowie bei der Strukturierung und Überarbeitung zu helfen. Gelegentlich wurden KI-Werkzeuge auch eingesetzt, um Ideen zu diskutieren, alternative Formulierungen vorzuschlagen und Unstimmigkeiten im Text zu identifizieren.

Es wurden keine KI-generierten Inhalte ohne kritische Prüfung und substanzielle Überarbeitung durch den Autor übernommen. Die konzeptionelle Entwicklung, die wissenschaftliche Argumentation, die Interpretation der Ergebnisse, die Auswahl der Literatur sowie alle in dieser Arbeit präsentierten Schlussfolgerungen liegen ausschließlich in der Verantwortung des Autors. Alle Literaturangaben, Zitate und sachlichen Aussagen wurden vor der Aufnahme unabhängig überprüft.

Abbildungsverzeichnis

Listings

Tabellenverzeichnis

12 Anhang

12.1 Technisches Handbuch

TECHNISCHES HANDBUCH

Handbuch zum (be)greifbaren Smart Home

Konzeption und Konstruktion einer modularen 12V-IoT-Lernumgebung für die Digitale Grundbildung





Sekundarstufe I • Unterrichts- und Laborumgebung • modulare Hardware • Digitale Grundbildung

Autor

Johannes Baischer

ERSCHEINUNGSJAHR

2026

Inhaltsverzeichnis

1 Einleitung **4**

2 Quick-Start Guide **5**

2.1 Minimalanforderungen . . . 5

2.2 Schritt für Schritt Anleitung . . . 5

3 Modellbau **6**

3.1 Materialien und Werkzeuge . . . 6

3.2 Design und Konstruktion . . . 6

4 Zugänge & Systemarchitektur **7**

4.1 Zugangsübersicht . . . 7

4.2 Verwendete Dienste und lokale Adressen . . . 7

4.3 Sinnvolle Arbeitsweise / Trouble Shooting . . . 8

5 Server Setup & Infrastruktur **9**

5.1 Ziel und Aufbau . . . 9

5.2 Xubuntu Installation . . . 9

5.3 SSH einrichten . . . 11

5.3.1 Passwortlose SSH Verbindung . . . 12

5.4 Remote Desktop konfigurieren . . . 13

5.4.1 Energieoptionen . . . 14

5.5 Docker Installation . . . 15

5.6 Router Setup und Netzstruktur . . . 16

5.6.1 A1-Router vorbereiten . . . 16

5.6.2 Internetfreigabe auf dem Xubuntu-Server . . . 16

5.7 Tailscale für Fernzugriff . . . 17

5.7.1 Technische Funktionsweise und Datenschutz . . . 18

6 Software Setup **19**

7 Modulverkabelung **20**

8 ESP32 Programme **21**

9 Anhang **i**



Todo list

- Remove TODO List . . . 3
- QR Code zum GitHub-Repository einfügen . . . 4
- Schnellanleitung für Profis mit nur Befehlen und Code-Snippets erstellen. . . . 5
- Materialliste und Werkzeugliste für ein einziges Modul . . . 5
- Befehlsliste und Code-Snippets für die Installation und Konfiguration der Lernumgebung . . . 5
- 03 Software Setup.md in LaTeX ausarbeiten . . . 19
- 02 Modulverkabelung.md in LaTeX ausarbeiten . . . 20
- 04 ESP32 Programme.md in LaTeX ausarbeiten . . . 21
- Bilder, Schaltpläne, Screenshots . . . i

Remove TODO List



1

Einleitung

Herzlich willkommen zum Handbuch für das *(be)greifbare Smart Home*. Dieses Dokument dient als praxisorientierter Leitfaden und ist als Begleitmaterial der vorliegenden Masterarbeit konzipiert. Es richtet sich an alle, die das vorgestellte lokale, sichere und vollständig transparente Smart-Home-System für den Schulunterricht von Grund auf selbst aufbauen möchten.

Im Gegensatz zu proprietären Lösungen großer Hersteller steht hier der Gedanke der digitalen Souveränität im Vordergrund. Die Technologie soll im wahrsten Sinne des Wortes „begreifbar“ und kontrollierbar bleiben. In den folgenden Kapiteln wird Schritt für Schritt durch den gesamten Einrichtungsprozess geführt. Dieser reicht von der initialen Vorbereitung der Hardware und dem Flashen des Betriebssystems über die Installation des Xubuntu-Servers bis hin zur sicheren Netzwerkkonfiguration (SSH) und der finalen Inbetriebnahme der Smart Home Dienste und Geräte.

Im Handbuch befindet sich auch ein Vorschlag für eine konkrete praktische Umsetzung als modulare Smart Home Rauminstallationen. Alle Schritte lassen sich aber auch unabhängig vom vorgeschlagenen Formfaktor durchführen.

Dank detaillierter Code-Listings, Terminal-Ausgaben und kommentierter Konfigurationsdateien ist dieses Handbuch so strukturiert, dass der Aufbau der Infrastruktur systematisch nachvollzogen und reproduziert werden kann. Alle Config-Skripts und Codebeispiele sind in einem eigenen GitHub-Repository verfügbar. Dort finden sich auch weiterführende Informationen, Updates und sowie die begleitende schriftliche Masterarbeit zum Thema.

Begleitendes GitHub-Repository

`https://github.com/Johannes-Baischer/Das--be-greifbare-Smart-Home`

QR Code zum GitHub-Repository einfügen

Wichtiger Hinweis zu Abbildungen und Quellcode

Soweit nicht in der jeweiligen Bild- oder Codeunterschrift explizit anders angegeben, handelt es sich bei allen Screenshots, Diagrammen und Terminal-Ausgaben in diesem Handbuch um eigene Darstellungen des Verfassers, die im Rahmen der praktischen Umsetzung dieser Masterarbeit erstellt wurden.



2

Quick-Start Guide

Schnellanleitung für Profis mit nur Befehlen und Code-Snippets erstellen.

Minimalanforderungen

Materialliste und Werkzeugliste für ein einziges Modul

Schritt für Schritt Anleitung

Befehlsliste und Code-Snippets für die Installation und Konfiguration der Lernumgebung



3

Modellbau

Materialien und Werkzeuge

Design und Konstruktion



4

Zugänge & Systemarchitektur

Zugangsübersicht

Sicherheit

Die folgende Übersicht zeigt die wichtigsten Zugänge und Dienste, die für den Aufbau und Betrieb der Smart-Home-Lernumgebung relevant sind. Die Logins sind hier nur als Beispiel dargestellt und müssen vor der Nutzung durch die tatsächlichen Zugangsdaten ersetzt werden.

Bereich	Zugriff IP/ID	Port	Benutzer	Passwort	Hinweis
WLAN	MMSE-Smart-Home-Wifi	–	–	***	Verbindung für Router, Server und Lernmodule
Router Config	10.0.0.138	80 / 443	admin	***	lokale Administration im Projekt-Netz
SSH	10.0.0.1/*mmse-smarthome*	22	mmse	***	Befehlszeile für Serververwaltung
RDP	10.0.0.1/*mmse-smarthome*	3389	mmse	***	GUI-Zugriff für Wartung und Demonstration
Ubuntu Server	SSH/RDP	22 / 3389	mmse	***	Benutzeraccount für Kamera-Einbundung
Home Assistant	10.0.0.1:8123	8123	mmse	***	lokale Bedienoberfläche für Smart-Home-Logik
TP-Link	–	–	Johannes.Baischer@nmmse.at	***	Zugang zur Konfiguration der Kamera
Kamera	–	–	mmse	***	für Gerätedokumentation und Monitoring

Verwendete Dienste und lokale Adressen

Die praktische Infrastruktur basiert auf einem klaren Muster: Das Projekt-Netz ist lokal und wird vom Server als Gateway und Router für die Lernumgebung genutzt. Dadurch bleiben die Endgeräte untereinander erreichbar, ohne dass sie aus dem Schulnetz herauslaufen müssen.

Server-Host

Der Basisserver läuft als Xubuntu-Host mit dem Namen `mmse-smarthome`. Die lokale Verbindung nutzt dabei in der Regel die Adresse `10.0.0.1` oder den Hostnamen im selben Netzwerk.

Router

Der A1-Router bleibt als Verwaltungs- und Verteilungsgerät im lokalen Netz aktiv, aber ohne DHCP-Server (diese Aufgabe übernimmt der Server-Host). Der Router fungiert also nach seiner Konfiguration als **Access Point** Seine Adresse bleibt `10.0.0.138`.



Home Assistant

Die Automationsoberfläche läuft in der Regel auf Port `8123`. Damit ist sie auf dem lokalen Netzwerk per Browser erreichbar, zum Beispiel über `http://10.0.0.1:8123`.

SSH und RDP

Für Administration und Fernwartung werden SSH auf Port `22` sowie Remote Desktop auf Port `3389` genutzt. Beide Dienste greifen auf denselben Server-Account zu.

Fernzugriff

Tailscale

Für externen Fernzugriff wird ein Overlay-VPN über Tailscale eingerichtet. Dadurch bleibt der Zugriff auch außerhalb des lokalen Netzes möglich, ohne Ports im Schulnetz freizugeben.

Sinnvolle Arbeitsweise / Trouble Shooting

- Kennzeichne die Zugangsdaten immer am Anfang der Projektphase, damit alle Beteiligten denselben Zugang verwenden.
- Nutze getrennte Benutzer für Server, Kamera und Home Assistant, um Fehlerquellen zu reduzieren.
- Verwende möglichst lokale Zugriffspfadnamen wie `mmse-smarthome` statt nur IP-Adressen, wenn du dich im selben Netzwerk befindest.
- Stelle sicher, dass alle Geräte nach dem Netzwerksetup einmal neu gestartet werden, damit sie die neue IP-Konfiguration übernehmen (besonders nach einmaligem DHCP Wechsel von Router auf Server-Host).

Internet Zugang

- Nachdem der A1-Router als Access-Point konfiguriert wird, kann der Server sich mit einem beliebigem zweiten Netzwerk verbinden, um **Internet Zugang** zu erhalten.



5

Server Setup & Infrastruktur

Ziel und Aufbau

Der Server bildet das zentrale Rückgrat der Smart-Home-Lernumgebung. Er stellt die Grundlage für Netzwerkzugang, Fernverwaltung, Containerisierung und zentrale Automationslogik bereit. In diesem Kapitel installierst und konfigurierst du die Infrastruktur schrittweise.

Working Principle

Ein Server muss in der Regel drei Dinge zuverlässig leisten: Zugriff auf das Internet, lokale Verteilung im Projekt-Netz und Remote-Verwaltung. Das Design legt dabei bewusst die Verantwortung auf den Xubuntu-Host, während der A1-Router nur noch als Netzwerkausgang und Access Point dient.

Xubuntu Installation

1. **Stelle sicher, dass das Servergerät keine wichtigen Daten enthält, da die Installation das System vollständig überschreibt.**
2. Lade das aktuelle **Xubuntu-24.04-Image** von der offiziellen Seite (https://xubuntu.org/release/24.04/) herunter und brenne es mit der Software **balenaEtcher** (https://etcher.balena.io/) auf einen USB-Stick.

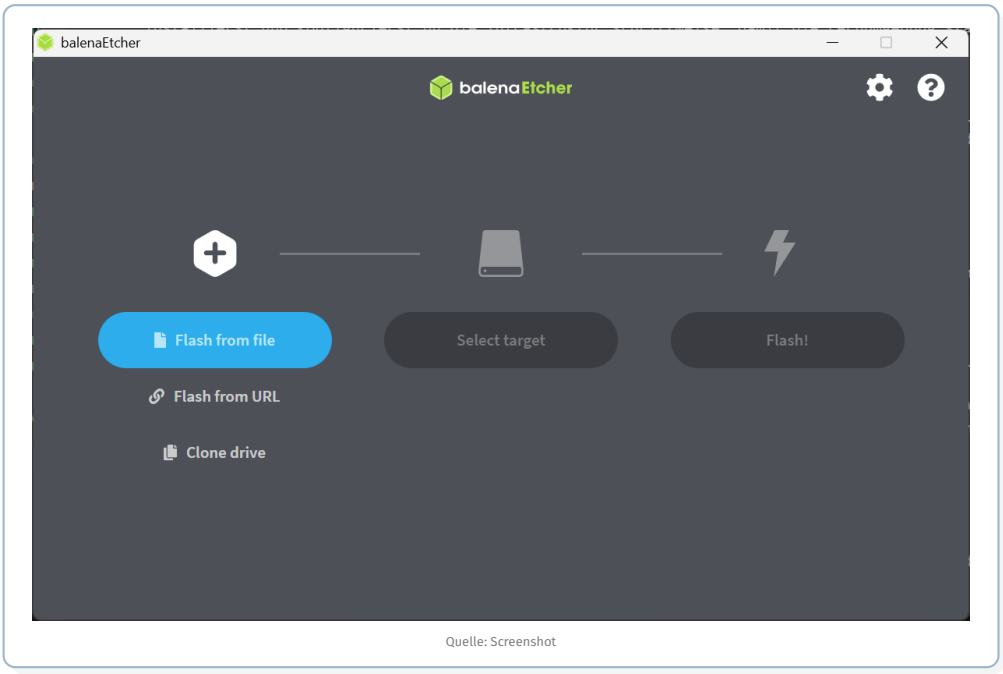


Abbildung 5.1: balenaEtcher: USB-Image auf Stick brennen

3. Schließe den USB-Stick an das Servergerät an und starte das System über das BIOS- bzw. Boot-Menü. Manche Hersteller verlangen ein ändern der Boot-Reihenfolge, sodass der USB-Stick als erstes Gerät



geladen wird.

Wie gelange ich ins BIOS bzw. Boot-Menü?

Je nach Hersteller und Modell des Geräts kann der Zugang unterschiedlich sein. Häufig sind die Tasten F2, F10, F12 oder ESC relevant. Beim Neustart des Geräts achte auf die entsprechende Anzeige, die den Zugang zum BIOS oder Boot-Menü beschreibt und drücke die passende Taste rechtzeitig. **Alle Herstellerhinweise findet man frei zugänglich im Internet.**

BIOS Menü

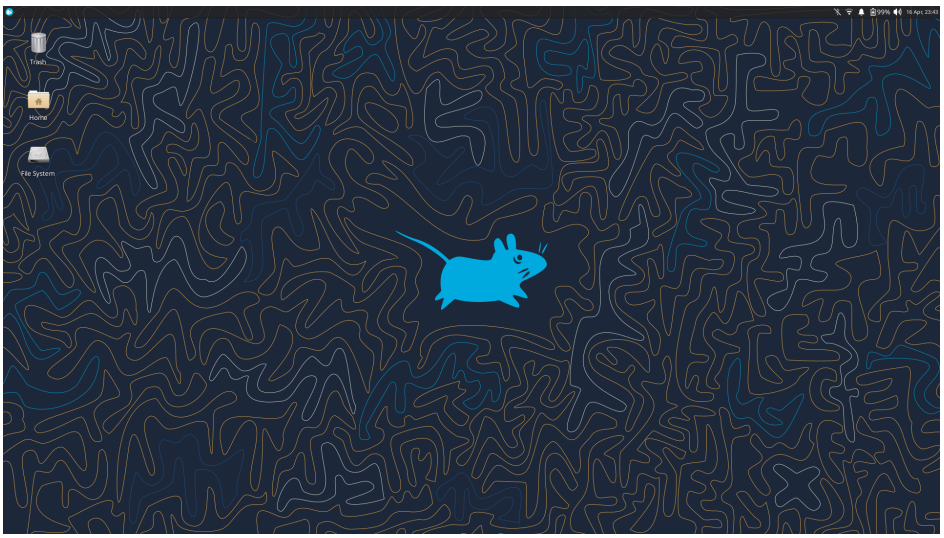
4. Wähle die Option *Try or install Xubuntu*.
5. Führe den Installationsprozess durch. Erstelle dabei einen passenden Benutzer mit einem sicheren Passwort und benenne das Gerät. Für den durchgeführten Testaufbau wählen wir als Benutzer das Kürzel der Schule `mmse` und als Gerätenamen `mmse-smarthome`.

Hinweis zur Xubuntu-Installation

Die Installation von Xubuntu ist ein einmaliger Schritt. Danach kann das System über die grafische Oberfläche oder per SSH verwaltet werden. Eine detaillierte Schritt-für-Schritt-Anleitung für die Installation von Xubuntu ist im Internet verfügbar, z. B. auf `https://ubuntu.com/desktop/docs/en/latest/tutorial/install-ubuntu-desktop/`.

Hinweis zu alten Geräten

Sollte ein sehr altes Gerät verwendet werden, kann es vorkommen, dass bestimmte Komponenten wie WLAN-Chipsätze nicht mehr unterstützt werden. In diesem sehr seltenen Fall kann es notwendig sein, auf ein aktuelleres Gerät auszuweichen oder Ausweichlösungen zu nutzen.
Der verwendete Toshiba Satellite Pro Laptop, wird mit seinem Broadcom-WLAN-Chipsatz nicht unterstützt. Hier wurde ein Edimax EW-7811UN USB-WLAN-Stick als Ersatz verwendet.
In diesem Schritt solltest du jedenfalls eine temporäre Internetverbindung über LAN oder WLAN sicherstellen, um die Installation und spätere Updates durchführen zu können. Diese Verbindung kann später wieder entfernt werden, sobald das lokale Projekt-Netzwerk korrekt eingerichtet ist.



Quelle: Screenshot

Abbildung 5.2: Erfolgreicher Xubuntu Startup

SSH einrichten

SSH ermöglicht den sicheren Zugriff auf den Server über die Kommandozeile. Damit kannst du das System auch von einem anderen Rechner aus verwalten, ohne direkt am Gerät zu sitzen.

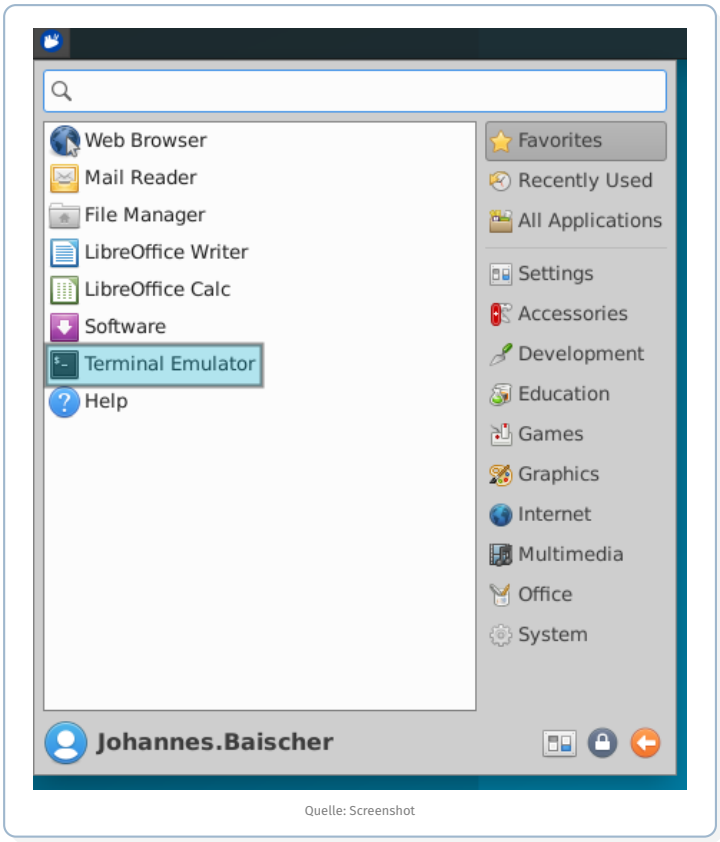


Abbildung 5.3: Terminal öffnen

Listing 5.1: SSH-Installation und Aktivierung auf dem Xubuntu-Server bash

```
sudo apt update
sudo apt install openssh-server -y
sudo systemctl enable --now ssh
sudo ufw allow ssh
sudo systemctl status ssh
hostname -I
```

Die im Listing 5.1 (SSH-Installation und Aktivierung auf dem Xubuntu-Server) zu sehenden Befehle bewirken beim Eingeben in ein geöffnetes Terminal Folgendes:

1. Installiert den aktuellsten OpenSSH-Server auf dem Xubuntu-Host.
2. Aktiviert den Dienst automatisch beim Systemstart.
3. Öffnet den SSH-Port in der Firewall und prüft den Verbindungsstatus.
4. Gibt die lokale IP-Adresse des Servers aus, die für die Verbindung von einem anderen Rechner benötigt wird.



SSH-Verbindung testen

Teste die Verbindung von einem anderen Rechner im gleichen Netzwerk mit dem Befehl `ssh <Benutzer>@<IP-Adresse>`. Dabei ersetzt du `<Benutzer>` durch den zuvor erstellten Benutzernamen und `<IP-Adresse>` durch die vom Server ausgegebene Adresse. Bestätige die Sicherheitsabfrage mit `yes` und gib das Passwort ein, um die Verbindung herzustellen. Danach kannst du den Server über die Kommandozeile steuern.

Passwortlose SSH Verbindung

Sollte die Verbindung regelmäßig genutzt werden, kann es sinnvoll sein, den Rechner, mit dem man sich auf den Server verbindet, als vertrauenswürdiges Gerät zu hinterlegen. Dadurch **entfällt die Eingabe des Benutzerpasswortes** bei jeder SSH-Verbindung. Dafür erstellen wir einmalig einen sog. *SSH-Key* auf unserem Windows-Rechner und übertragen den öffentlichen Teil des Schlüssels auf den Server. Danach authentifiziert sich der Rechner ab der nächsten SSH-Verbindung automatisch durch seinen privaten Schlüssel, ohne dass wir das eigentliche Passwort eingeben müssen.

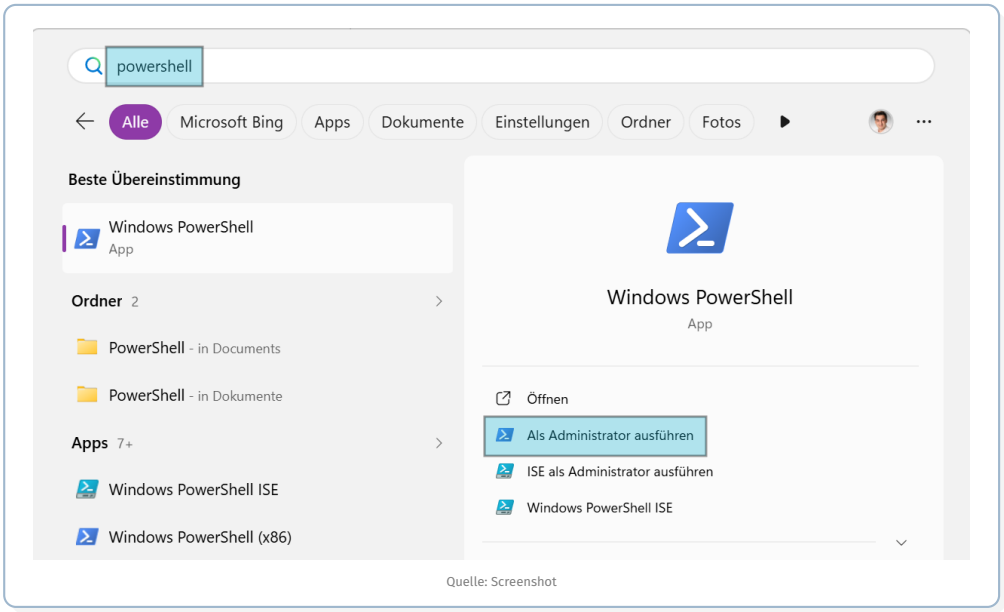


Abbildung 5.4: PowerShell in Windows öffnen

Listing 5.2: PowerShell-Befehle für SSH-Keys in Windows bash

```bash
ssh-keygen -t ed25519

type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh mmse@mmse-smarthome "
    mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/
    authorized_keys && chmod 600 ~/.ssh/authorized_keys"

ssh mmse@mmse-smarthome
```




Hinweis zu Benutzername und Hostname

Der Benutzername und Hostname des Servers müssen in den Befehlen angepasst werden.

SSH-Keys für MacOS und Linux

Auf MacOS und Linux ist die Vorgehensweise ähnlich. Auch dort können SSH-Keys erstellt und auf den Server übertragen werden. Die genauen Befehle findest du unter `https://man.openbsd.org/ssh-keygen.1`.

Remote Desktop konfigurieren

Mit Remote Desktop kannst du zusätzlich zur Fernsteuerung über die Kommandozeile den grafischen Desktop des Xubuntu-Servers über das Netzwerk ansteuern. Das ist besonders hilfreich, wenn du GUI-basierte Konfigurationen oder Wartungsarbeiten durchführst, bei denen das grafische Einstellungsmenü hilfreich sein könnte.

1. Terminal öffnen
2. Aktualisiere das System und installiere anschließend den RDP-Server `xrdp`.
3. Konfiguriere `xrdp` so, dass es die XFCE-Oberfläche verwendet.
4. Vergib die nötigen Zertifikatsrechte und starte den Dienst neu.
5. Öffne den RDP-Port in der Firewall und prüfe mit `hostname -I` die Serveradresse.

Xubuntu basiert auf XFCE

Listing 5.3: Xubuntu-Setup für Remote Desktop bash

```bash
sudo apt update && sudo apt upgrade -y
sudo apt install xrdp -y
echo xfce4-session > ~/.xsession
sudo adduser xrdp ssl-cert
sudo systemctl restart xrdp
sudo systemctl enable xrdp
sudo ufw allow 3389/tcp
hostname -I
```

Vor der Remote Desktop Verbindung

Wenn der Laptop lokal weiter aktiv ist, kann der gleiche Benutzer nicht gleichzeitig per Remote Desktop angemeldet sein. **Melde dich deshalb immer erst lokal ab, bevor du eine neue Remote-Verbindung startest.** Das verhindert schwarze Bildschirme und Abbrüche.

Verwende für die Remote-Verbindung den Windows-Dienst *Remotedesktopverbindung* und trage dort den Hostnamen oder die IP-Adresse des Servers ein. Danach kannst du dich mit dem zuvor erstellten Benutzer anmelden.



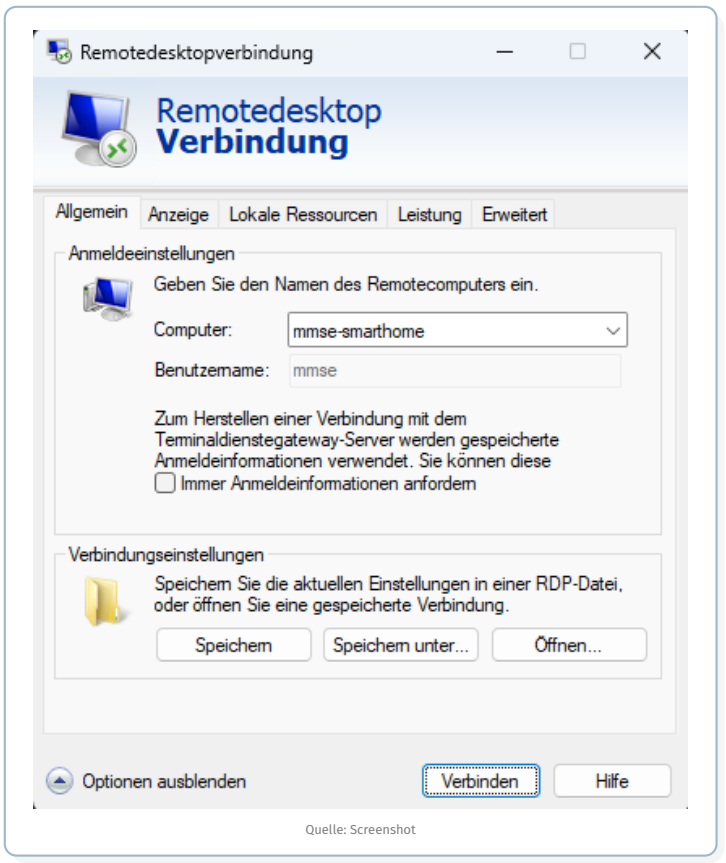


Abbildung 5.5: Windows-Remote-Desktop

Energieoptionen

Falls es sich beim Servergerät um einen Laptop handelt, müssen die systemweiten Energieeinstellungen angepasst werden. Dadurch wird verhindert, dass das Gerät nach inaktiver Zeit oder bei geschlossenem Deckel in den Energiesparmodus wechselt. Das ist besonders wichtig, wenn **kein Benutzer lokal angemeldet ist**, da sonst die USB-Stromversorgung (z. B. für den WLAN-Stick) gekappt wird.

Listing 5.4: Energieoptionen für Laptop-Server — bash

```
sudo nano /etc/systemd/logind.conf
# HandleLidSwitch=ignore
# HandleLidSwitchExternalPower=ignore
# HandleLidSwitchDocked=ignore

sudo systemctl restart systemd-logind
sudo systemctl mask sleep.target suspend.target hibernate.target
    hybrid-sleep.target
```

Die Anpassung erfolgt wie in Listing 5.4 (Energieoptionen für Laptop-Server) in zwei Schritten:

1. **Aktion beim Schließen des Deckels (Lid Switch) deaktivieren:**
   Öffne die Systemd-Login-Konfiguration im Terminal mit `sudo nano /etc/systemd/logind.conf`. Suche dort nach den folgenden Zeilen, **entferne das #-Zeichen am Zeilenanfang** und setze den Wert jeweils auf `ignore`:

- `HandleLidSwitch=ignore`
- `HandleLidSwitchExternalPower=ignore`
- `HandleLidSwitchDocked=ignore`

Speichere die Datei (`Strg+O`, `Enter`, `Strg+X`) und starte den Dienst mit `sudo systemctl restart systemd-logind` neu (*kann die Session ggf. kurz unterbrechen*).

2. **Systemweiten Standby und Ruhezustand komplett deaktivieren:**
   Da der Laptop dauerhaft als Server betrieben wird, werden sämtliche Energiespar-Targets auf Systemebene blockiert:
   - `sudo systemctl mask sleep.target suspend.target hibernate.target hybrid-sleep.target`

   Dadurch wird verhindert, dass das Betriebssystem in den Schlafmodus wechselt.

Energiespar-Sperre aufheben

Falls die Blockierung der Energiespar-Targets später wieder aufgehoben werden soll, kann dies mit dem Befehl `sudo systemctl unmask sleep.target suspend.target hibernate.target hybrid-sleep.target` rückgängig gemacht werden.

Docker Installation

Docker stellt eine Container-Laufzeitumgebung bereit, in der Home Assistant und weitere Dienste voneinander getrennt ausgeführt werden können. Für dieses Handbuch wird die Docker Engine direkt auf Xubuntu installiert. Docker Desktop ist für diesen Serveraufbau nicht vorgesehen, da es eine eigene Desktop-Umgebung voraussetzt.

Voraussetzung für die Containerumgebung

Verwende eine Docker Engine ab Version 23.0.0. Die Installation erfolgt über die Xubuntu-Paketverwaltung und umfasst Docker Compose v2. Die benötigte Version kann nach der Installation mit `docker –version` geprüft werden.

1. Aktualisiere zunächst die Paketquellen und installiere die Docker Engine gemeinsam mit Docker Compose v2.
2. Aktiviere den Docker-Dienst, damit er direkt gestartet wird und künftig automatisch mit dem System hochläuft.
3. Ergänze den aktuell angemeldeten Benutzer in der Docker-Gruppe. Dadurch können Containerbefehle später ohne sudo ausgeführt werden.

Listing 5.5: Installation von Docker Engine und Docker Compose bash

```bash
sudo apt update
sudo apt install docker.io docker-compose-v2 -y
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
```



Änderung der Gruppenrechte übernehmen

Die neue Gruppenzugehörigkeit wird erst nach einer erneuten Anmeldung wirksam. Melde dich deshalb nach der Installation einmal ab und wieder an. Anschließend lässt sich die Installation beispielsweise mit `docker run hello-world` überprüfen.

Router Setup und Netzstruktur

Aufbau des Projekt-Netzes

Das Projekt-Netzwerk wird bewusst vom übrigen Netzwerk getrennt. Der Xubuntu-Server übernimmt dabei die Funktion des Gateways. Er bezieht den Internetzugang über WLAN und stellt über seine Ethernet-Schnittstelle ein eigenes lokales Netz bereit. Der verwendete A1-Router wird nicht mehr als Router oder DHCP-Server eingesetzt, sondern dient nur noch als Switch und Access Point.

Der Xubuntu-Server verbindet zwei Netze miteinander. Über WLAN erreicht er das Internet, über die Ethernet-Schnittstelle stellt er das lokale Netz `10.0.0.0/24` bereit. Die Adresse `10.0.0.1` ist dabei das Gateway für die angeschlossenen Geräte und gleichzeitig die Adresse des Servers selbst.

A1-Router vorbereiten

Bevor der Router mit dem Server verbunden wird, muss sein eigener Netzwerkdienst deaktiviert werden. Andernfalls könnten der Router und der Xubuntu-Server gleichzeitig IP-Adressen vergeben und es käme zu Konflikten.

Abweichung je nach Hersteller

1. Verbinde einen Rechner mit dem A1-Router und öffne dessen Weboberfläche unter `10.0.0.138`.
2. Deaktiviere den DHCP-Server im Bereich für das Heimnetzwerk bzw. LAN.
3. Entferne vorhandene statische IP-Bindungen oder Reservierungen.
4. Belasse die Routeradresse bei `10.0.0.138`. Sie liegt damit weiterhin im später verwendeten lokalen Subnetz und bleibt erreichbar.

Internetfreigabe auf dem Xubuntu-Server

Verbinde den Server zunächst über WLAN mit dem Internet. Für die Ethernet-Verbindung wird anschließend der NetworkManager-Modus *shared* verwendet. In diesem Modus übernimmt der Xubuntu-Server die lokale Adressvergabe und stellt den angeschlossenen Geräten den eigenen Internetzugang über die WLAN-Verbindung bereit.

Listing 5.6: Internet-Sharing und lokales Subnetz auf dem Xubuntu-Host bash

```
nmcli connection show
sudo nmcli connection modify 'Wired connection 1' ipv4.addresses
    10.0.0.1/24 ipv4.method shared
sudo nmcli connection down 'Wired connection 1' && sudo nmcli
    connection up 'Wired connection 1'
ip a
```

Die Befehle im Listing 5.6 führen die Konfiguration in vier Schritten durch:

1. Mit `nmcli connection show` wird der Name der Ethernet-Verbindung ermittelt. Je nach Installation kann er beispielsweise `Wired connection 1` lauten.



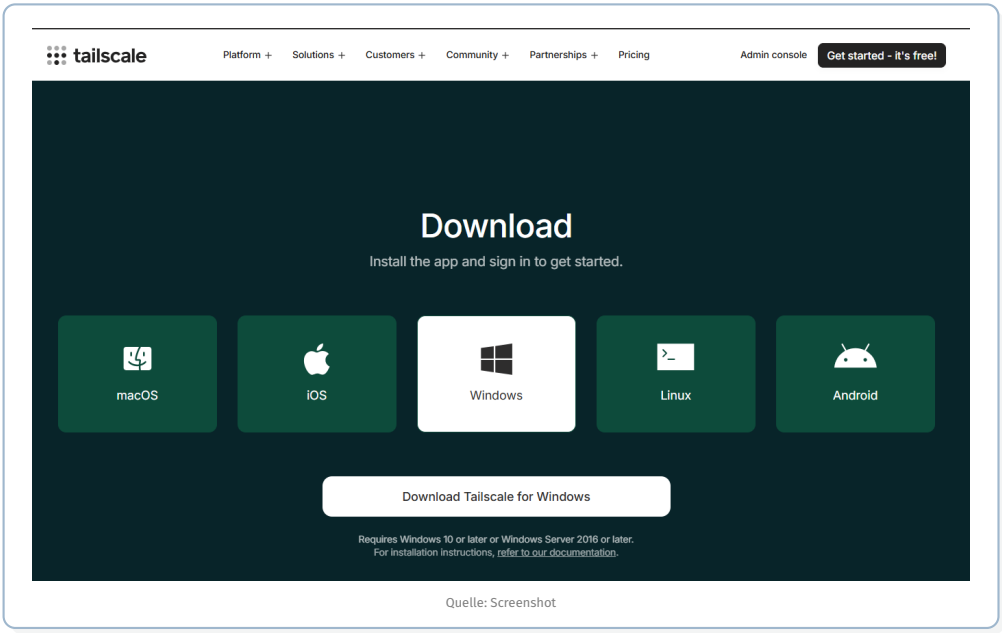
2. Die Verbindung erhält die Adresse `10.0.0.1/24` und wird in den gemeinsamen NetworkManager-Modus versetzt.
3. Durch das erneute Starten der Verbindung werden die Änderungen übernommen.
4. Mit `ip a` wird geprüft, ob die Ethernet-Schnittstelle die Adresse `10.0.0.1` verwendet.

Geräte mit dem Projekt-Netz verbinden

Verbinde den Server per LAN mit LAN-Port 1 des A1-Routers und verbinde danach die Smart-Home-Geräte per WLAN mit dem Router. Starte die Endgeräte anschließend neu, damit sie ihre Adresse vom Xubuntu-Server beziehen. Home Assistant ist nach erfolgreicher Installation (siehe Kapitel 6 *Software Setup*) im lokalen Netz über `http://10.0.0.1:8123` erreichbar.

Tailscale für Fernzugriff

Das lokale Projekt-Netz ist absichtlich nicht direkt aus dem Internet erreichbar. Für die Fernwartung wird deshalb Tailscale als Overlay-Netzwerk eingesetzt. Damit kann ein autorisiertes Privatgerät eine verschlüsselte Verbindung zum Xubuntu-Server herstellen, ohne Ports im Schul- oder Heimnetz des Routers freigeben zu müssen, was aufgrund von Sicherheitsrichtlinien oft ohnehin nicht möglich ist.



Quelle: Screenshot

Abbildung 5.6: Tailscale herunterladen

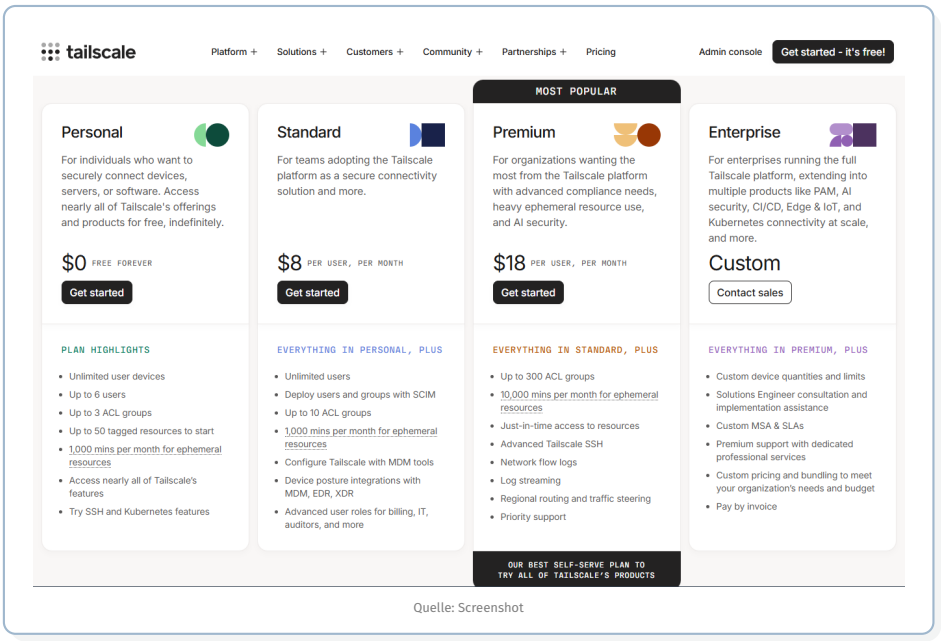
1. Installiere Tailscale auf dem Xubuntu-Server und starte anschließend die Anmeldung.
2. Öffne den im Terminal ausgegebenen Link und authentifiziere den Server im vorgesehenen Tailscale-Konto.
3. Installiere Tailscale zusätzlich auf dem privaten Laptop oder Smartphone und melde auch dieses Gerät im selben Konto an.
4. Ermittle die Tailscale-IP des Xubuntu-Servers und rufe Home Assistant (nach erfolgreicher Anmeldung) über diese Adresse und den Port 8123 auf.



Listing 5.7: Installation und Anmeldung von Tailscale auf dem Server bash

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
sudo tailscale status
```

Die Anmeldung und der Verbindungsstatus können anschließend über die Tailscale-Oberfläche bzw. mit `tailscale status` kontrolliert werden. Die konkrete Tailscale-IP ist kontospezifisch und kann sich ändern, sollte daher nicht fest in irgendwelchen Konfigurationen hinterlegt werden.



Quelle: Screenshot

Abbildung 5.7: Tailscale Pricing

Technische Funktionsweise und Datenschutz

Tailscale verwendet das WireGuard-Protokoll für die verschlüsselte Kommunikation zwischen den angemeldeten Geräten. Die zentrale Steuerungsebene vermittelt Identitäten, öffentliche Schlüssel und Verbindungsinformationen. Die Nutzdaten werden nach Möglichkeit direkt zwischen den Endgeräten übertragen.

Ende-zu-Ende verschlüsselt

Direkte Verbindung und Relay-Fallback

Für den Verbindungsaufbau versucht Tailscale zunächst, eine direkte Verbindung durch NAT-Traversal herzustellen. Wenn die beteiligten Netzwerke dies nicht zulassen, kann ein verschlüsselter DERP-Relay als Vermittler verwendet werden. Auch in diesem Fall bleibt der eigentliche Datenverkehr Ende-zu-Ende verschlüsselt.

Die privaten Schlüssel verbleiben auf den jeweiligen Endgeräten. Der Tailscale-Koordinationsdienst verwaltet daher die für den Verbindungsaufbau notwendigen Metadaten, kann den verschlüsselten Nutzdatenstrom jedoch nicht entschlüsseln. Der Zugriff auf Home Assistant ist zusätzlich auf Geräte beschränkt, die im eigenen Tailscale-Netz autorisiert wurden.

Standardmäßig wird dabei kein vollständiger Internetverkehr durch den VPN-Tunnel geleitet. Für dieses Projekt ist nur der Zugriff auf den Server und gegebenenfalls auf das Projekt-Netz erforderlich.



6

Software Setup

03 Software Set-up.md in LaTeX ausarbeiten



7

Modulverkabelung

02 Modulverkabelung.md in LaTeX ausarbeiten



8

ESP32 Programme

04 ESP32 Programme.md in LaTeX ausarbeiten



9

Anhang

Bilder, Schaltpläne, Screenshots



Abbildungsverzeichnis

5.1 balenaEtcher: USB-Image auf Stick brennen 9
5.2 Erfolgreicher Xubuntu Startup 10
5.3 Terminal öffnen 11
5.4 PowerShell in Windows öffnen 12
5.5 Windows-Remote-Desktop 14
5.6 Tailscale herunterladen 17
5.7 Tailscale Pricing 18



Listings

5.1 SSH-Installation und Aktivierung auf dem Xubuntu-Server 11
5.2 PowerShell-Befehle für SSH-Keys in Windows 12
5.3 Xubuntu-Setup für Remote Desktop 13
5.4 Energieoptionen für Laptop-Server 14
5.5 Installation von Docker Engine und Docker Compose 15
5.6 Internet-Sharing und lokales Subnetz auf dem Xubuntu-Host 16
5.7 Installation und Anmeldung von Tailscale auf dem Server 17



Tabellenverzeichnis

